

Сеточные рекуррентные сети: параллельные колонки памяти для частично наблюдаемых сред

Анонимная подача

Аннотация

Последовательное принятие решений в условиях частичной наблюдаемости требует от агентов поддержания структурированной и избирательной памяти на длинных горизонтах. Стандартные рекуррентные архитектуры (GRU, LSTM) опираются на единый однородный скрытый вектор, не предоставляя структурного механизма для специализации ролей памяти. Сеточные рекуррентные архитектуры (MD-RNN, Grid LSTM) организуют ячейки в двумерную структуру, однако используют **фиксированные** позиционные связи между ячейками и не исследовали модульную замену механизмов памяти и коммуникации. Мы переосмысливаем парадигму Grid RNN как **модульный фреймворк** с двумя сменными компонентами: рекуррентной ячейкой колонки (GRU / LRU / быстрые веса) и уровнем межколоночной передачи сообщений (масштабированное внимание / поиск Хопфилда). Межколоночное внимание с обучаемыми идентификаторами колонок обеспечивает **контентно-ориентированный** обмен и спонтанную специализацию ролей без явного надзора. Мы оцениваем три конфигурации фреймворка на трёх взаимодополняющих бенчмарках: задаче ассоциативной памяти Store-Distract-Query (SDQ), наборе задач частично наблюдаемого обучения с подкреплением MIKASA/POPGym и посимвольном языковом моделировании. Конфигурация **HopfieldGridLRU** (ячейки LRU + поиск Хопфилда) достигает 96.7% точности на SDQ-Hard (против $\approx 50\%$ у GRU); **GridRNN-EMA** (GRU + быстрые веса с EMA-гейтом удивления) достигает суммарного возврата ≈ 0.95 на POPGym RepeatFirst. При 2.1M параметрах GridHarmonic достигает 1.68 BPC на text8. Результаты демонстрируют устойчивые улучшения от модульной сеточной структуры во всех трёх задачах.

Введение

Способность избирательно хранить и извлекать информацию во времени является ключевой для широкого класса последовательных задач: агенты обучения с подкреплением в частично наблюдаемых средах должны помнить увиденные объекты и связывать стимулы с отложенными последствиями; языковые модели должны поддерживать согласованный контекст на сотнях символов; задачи ассоциативной памяти требуют явного связывания пар ключ-значение среди множества отвлекающих факторов. Решение этих задач одновременно требует памяти с несколькими специализированными ролями, работающими на разных временных масштабах.

Стандартные рекуррентные архитектуры — GRU (Cho и др., 2014) и LSTM (Hochreiter & Schmidhuber, 1997) — решают это с помощью единого вектора скрытого состояния, обновляемого на каждом шаге. При всей эффективности этот **однопоточный** дизайн не обеспечивает структурного механизма специализации: все временные масштабы и функции памяти конкурируют за одни и те же измерения состояния. Последовательная модель должна одновременно быть краткосрочным буфером, долгосрочным интегратором

и контент-адресуемым хранилищем, однако её архитектура не предоставляет структурного разделения этих ролей.

Предшествующие работы двигались в нескольких направлениях. **Иерархические** архитектуры (HM-RNN (Chung и др., 2017), Fast-Slow RNN (Mujika и др., 2017), HGRN2 (Qin и др., 2024)) выстраивают несколько временных масштабов **вертикально**, назначая каждому слою разное временное разрешение. Это улучшает моделирование дальних зависимостей, но не поддерживает **латеральную** специализацию: каждый слой по-прежнему обрабатывает единый поток без взаимодействия с соседями. **Независимые параллельные потоки** (IndRNN (Li и др., 2018)) запускают несколько независимых рекуррентных нейронов без обмена между ними — специализация невозможна. Подходы **расширения памяти** (быстрые веса (Ba и др., 2016), сети Хопфилда (Ramsauer и др., 2021), NTM/DNC (Graves и др., 2014; 2016), энграммы (Szelogowski, 2025)) добавляют внешние ассоциативные структуры к **единственному** потоку, не меняя топологию. Слотовые линейные архитектуры (Gated Slot Attention (Zhang и др., 2024)) вводят несколько слотов памяти с гейтированной линейной рекуррентностью, однако

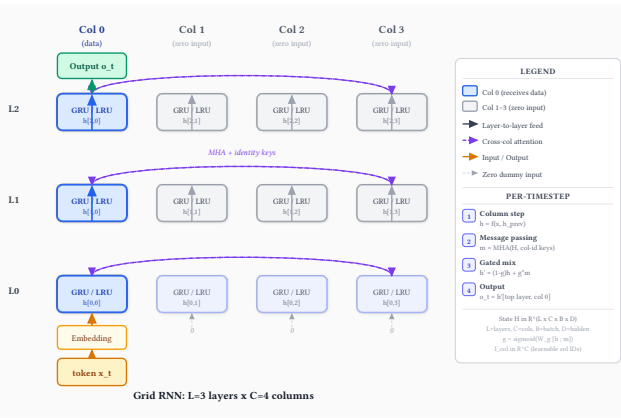


Рис. 1. Архитектура Grid RNN. Ячейки организованы в сетку $L \times C$. На каждом шаге каждая колонка независимо обрабатывает свой вход (колонка 0 получает эмбединг токена; остальные получают ноль), затем все колонки обмениваются информацией через многоголовочное внимание с обучаемыми идентификаторами колонок. Разные колонки сходятся к разным ролям памяти без явного надзора.

без явного механизма специализации через идентификаторы слотов.

Мы переосмысливаем парадигму Grid RNN — двумерных рекуррентных сетей (восходящих к MD-RNN (Graves и др., 2007) и Grid LSTM (Kalchbrenner и др., 2016)) — как **модульный фреймворк** с двумя сменными компонентами. Grid RNN поддерживает двумерную матрицу состояний $H \in \mathbb{R}^{L \times C \times B \times H}$, где L слоёв обеспечивают глубину, а C колонок работают **параллельно** (см. Рис. 1). В отличие от предшественников, использовавших фиксированные позиционные связи, наш фреймворк вводит **контентно-ориентированный** обмен через многоголовочное внимание с обучаемыми идентификаторами колонок — это позволяет колонкам **спонтанно специализироваться** без явного надзора.

Ключевое свойство фреймворка — модульность: небольшой заменой компонентов модель адаптируется к разным доменам. Мы исследуем три конфигурации:

- **GridRNN** (ячейки GRU + масштабированное внимание): сильный базовый вариант;
- **HopfieldGridLRU**: ячейки LRU (Orvieto и др., 2023) с переклоночными спектральными радиусами и поиском Хопфилда (Ramsauer и др., 2021) — для ассоциативной памяти;
- **GridRNN-EMA**: ячейки GRU с быстрыми весами (Ba и др., 2016) и EMA-гейтом удивления — для избирательной памяти в частично наблюдаемых средах.

Основные вклады:

1. **Модульный фреймворк Grid RNN**: двумерная сетка $L \times C$ с контентно-ориентированным межколоночным вниманием, обеспечивающим структурную специализацию. В отличие от IndRNN (Li и др., 2018) (независимые потоки без обмена) и NTM/DNC (Graves и др., 2014) (единый контроллер + внешняя память) — специализация возникает **внутри** рекуррентной динамики через управляемый обмен.
2. **HopfieldGridLRU**: достигает **96.7% точности** на SDQ-Hard, превосходя однопоточный GRU более чем в $6 \times$ по метрике Acc++.
3. **GridRNN-EMA**: достигает суммарного возврата ≈ 0.95 на POPGym RepeatFirst, приближаясь к оптимуму, с избирательной записью с гейтом удивления.
4. **Систематическое сравнение** на трёх бенчмарках (SDQ, POPGym/MIKASA, языковое моделирование) с устойчивым улучшением над однопоточными базовыми линиями.

Связанные работы

Многомерные и сеточные RNN.. Идея организации рекуррентных ячеек в двумерную структуру исследовалась ранее. Graves et al. ввели многомерные RNN (MD-RNN) (Graves и др., 2007), применяющие отдельные рекуррентные соединения вдоль каждого пространственного измерения — для субтитрования изображений и 2D-последовательностей. Grid LSTM (Kalchbrenner и др., 2016) расширил это до глубоких стеков, маршрутизируя память и скрытое состояние одновременно по осям глубины и времени. Эти работы устанавливают сеточную топологию, однако используют **фиксированные** позиционные соединения вместо **обученного** контентно-ориентированного внимания и ориентированы на 2D пространственные последовательности, а не на общее моделирование последовательностей. Наш фреймворк вводит **латеральное внимание с обучаемыми идентификаторами колонок** и исследует модульную замену механизмов памяти — аспект, отсутствующий в предшественниках.

Параллельные потоки без межпоточного общения.. IndRNN (Li и др., 2018) декоррелирует нейроны внутри каждого слоя, используя Адамарово произведение вместо матрицы переходов, и стекует несколько таких слоёв. Это устраняет взрыв/затухание градиентов и позволяет строить очень глубокие сети, но каждый нейрон полностью независим — никакой боковой коммуникации и специализации нет. Наш фреймворк добавляет управляемый обмен через уровень внимания именно там, где IndRNN его лишён.

Слотовые линейные архитектуры (Gated Slot Attention (Zhang и др., 2024)) поддерживают несколько контекстных слотов с гейтированной линейной рекур-

рентностью и softmax-чтением между слотами. GSA предлагает иной путь к специализации: через контекстно-зависимое считывание памяти, а не через точную топологию с идентификаторами. Наш подход дополняет GSA: сетка фиксирует пространственное разделение потоков, тогда как GSA максимизирует ёмкость в пределах одного потока.

Иерархические многомасштабные RNN. HM-RNN (Chung и др., 2017) обучает дискретные событийные границы для разделения быстрых и медленных вычислений в вертикальной иерархии. Fast-Slow RNN (Mujika и др., 2017) поддерживает две разединённые сети с фиксированным периодом связи. HGRN (Qin и др., 2023) и HGRN2 (Qin и др., 2024) вводят обучаемый гейт β для управления удержанием состояния на каждом слое линейного RNN, достигая конкурентоспособной производительности с избирательными пространственно-временными моделями. Все эти подходы расширяют **глубину** для создания иерархий временных масштабов, но каждый слой остаётся единым последовательным потоком без боковой коммуникации. Grid RNN дополняет вертикальную глубину **горизонтальными** колонками, связанными через внимание. Эмпирически 4 колонки с 3 слоями превосходят 2 колонки с 1 слоем при том же бюджете параметров на +42 п.п. по Acc++ на SDQ-Hard.

Расширение памяти: внешняя и внутрисетевая. Два подхода к дополнению рекуррентной памяти. **Внешние контроллеры памяти** — NTM (Graves и др., 2014) и DNC (Graves и др., 2016) — оснащают единственный контроллер RNN дифференцируемой внешней матрицей памяти с операциями чтения/записи на основе content-адресации. Они мощны, но разделяют память и контроллер топологически — специализация реализуется через головы чтения/записи, а не через структуру самого рекуррентного блока. **Внутрисетевые** механизмы — быстрые веса (Ba и др., 2016) (Хебб через внешние произведения), линейные Трансформеры как программисты быстрых весов (Schlag и др., 2021), ассоциативный LSTM (Danilhelka и др., 2016), современные сети Хопфилда (Ramsauer и др., 2021), нейроны-энграммы (Szelogowski, 2025) — встраивают ассоциативную память в **одиночный** рекуррентный поток. Каждый из них добавляет один специализированный компонент к однопоточному каркасу без изменения топологии. В нашем фреймворке ассоциативные механизмы инстанцируются **поколочно**: уровень сообщений Хопфилда маршрутизирует информацию между колонками, позволяя разным колонкам специализироваться на ролях чтения и записи. Это объединяет идеи внутрисетевой памяти с пространственным разделением потоков.

Память для частично наблюдаемого ОС. DRQN (Hausknecht & Stone, 2015) расширяет DQN с помощью LSTM для работы в условиях частичной наблюдае-

Конфигурация	Ячейка колонки	Уровень сообщений	Домен
GridRNN	GRU	Scaled dot-product	общий
HopfieldGridLRULRU	(2D спектр)	Hopfield search	SDQ / LM
GridRNN-EMA	GRU + fast weights	Scaled dot-product	POMDP RL
GridHarmonic	LRU + surprise EMA	Scaled dot-product	LM

Таблица 1. Конфигурации фреймворка Grid RNN. Каждая конфигурация получается заменой ячейки колонки или уровня межколоночных сообщений.

мости. Mnih et al. (Mnih и др., 2016) используют LSTM-политики в A3C. POPGym (Morad и др., 2023) систематически тестирует память в ОС по четырём типам — объектному, последовательному, ёмкостному и пространственному. MIKASA (Cherepanov и др., 2025) расширяет эту таксономию более богатыми вариантами задач. Эти бенчмарки неизменно показывают, что стандартный LSTM/GRU застревает значительно ниже теоретического максимума на объектных задачах типа RepeatFirst. GridRNN-EMA устраняет этот разрыв с помощью избирательного механизма записи с гейтом удивления, достигая почти оптимальной производительности на RepeatFirst.

Линейные рекуррентные единицы. Orvieto et al. (Orvieto и др., 2023) (LRU) показывают, что линейная рекуррентность с диагональными комплексными матрицами переходов достигает конкурентоспособной производительности при надлежащей инициализации спектральных радиусов. Мы используем ячейки LRU как единицы колонок в HopfieldGridLRU и GridHarmonic, назначая каждой паре колонка-слой отдельный $r_{\max}^{\{l,c\}}$, интерполированный по 2D-сетке от 0.3 до 0.999.

Метод

Фреймворк: два сменных интерфейса

Grid RNN определяет двумерную рекуррентную структуру из L слоёв и C колонок. Фреймворк параметризован **двумя независимыми компонентами**, каждый из которых может быть заменён:

Эта модульность отличает фреймворк от предшественников: MD-RNN (Graves и др., 2007) и Grid LSTM (Kalchbrenner и др., 2016) имеют фиксированные внутренние соединения, а NTM (Graves и др., 2014) / DNC (Graves и др., 2016) наращивают единственный поток внешней памяти.

Структура сетки

Grid RNN поддерживает трёхмерный тензор состояний $H_t^l \in \mathbb{R}^{C \times B \times H}$ на каждом слое l , где C — число колонок, B — размер батча, H — размерность скрытого состояния. На каждом шаге t прямой проход выполняется **послойно** в два этапа.

Шаг колонки.. Каждая колонка c независимо принимает свою рекуррентную ячейку f_θ^c :

$$h_t^{\{l,c\}} = f_\theta^c(x_t^{\{l,c\}}, h_{t-1}^{\{l,c\}})$$

Для слоя $l = 0$: колонка 0 получает вложенный вход $e_t \in \mathbb{R}^D$; колонки $c > 0$ получают нулевой фиктивный вход размерности 1. Для слоёв $l > 0$: все колонки получают соответствующее состояние предыдущего слоя как вход.

Передача сообщений между колонками.. После шага колонки все колонки обмениваются информацией через многоголовочное внимание:

$$Q_t = H_t^l + I_{\text{col}} \quad K_t = H_t^l + I_{\text{col}}$$

$$M_t^l = \text{MHA}(Q_t, K_t, H_t^l)$$

где $I_{\text{col}} \in \mathbb{R}^{C \times 1 \times H}$ — обучаемые поколоночные идентификаторные вложения, транслируемые по батчу. Выходная проекция MHA инициализируется близко к нулю, так что начальные сообщения незначительны — обучение начинается с эффективной однопоточной базовой линии.

Гейтованное смешивание.. Каждая колонка решает, насколько включать агрегированное сообщение:

$$g_t^{\{l,c\}} = \sigma\left(W_g \begin{bmatrix} h_t^{\{l,c\}}; m_t^{\{l,c\}} \end{bmatrix}\right)$$

$$h_t'^{\{l,c\}} = \left(1 - g_t^{\{l,c\}}\right) \odot h_t^{\{l,c\}} + g_t^{\{l,c\}} \odot m_t^{\{l,c\}}$$

Этот **постсообщенческий** дизайн (сначала шаг GRU, затем внимание) позволяет каждой колонке вычислить новое состояние независимо, прежде чем рассматривать соседей.

Выход.. Для ОС и ассоциативных задач выходом является состояние верхнего слоя ($l = L$) колонки 0: $o_t = h_t'^{\{L,0\}}$. Для языкового моделирования усредняем по всем колонкам.

Спектральная памятная сетка (HopfieldGridLRU)

Для задач с долгосрочными ассоциациями (SDQ) и посимвольного моделирования мы заменяем ячейки GRU на линейные рекуррентные единицы (LRU) (Orvieto и др., 2023) и заменяем уровень сообщений на ассоциативный поиск Хопфилда (Ramsauer и др., 2021).

Ячейки LRU.. Каждая пара колонка-слой (l, c) поддерживает комплекснозначное диагональное состояние $h_t \in \mathbb{C}^H$:

$$h_t = \Lambda^{\{l,c\}} \odot h_{t-1} + Bx_t$$

где $\Lambda^{\{l,c\}} = \text{diag}(\exp(-\exp(\nu) + i \exp(\theta)))$ обеспечивает $|\Lambda^{\{l,c\}}| \in (r_{\min}, r_{\max}^{\{l,c\}})$ по построению.

2D спектральная сетка.. Каждая пара колонка-слой получает отдельный спектральный радиус:

$$r_{\max}^{\{l,c\}} = r_{\min_{\text{col}}} + (r_{\text{base}}^l - r_{\min_{\text{col}}}) \cdot \frac{c}{C-1}$$

с $r_{\min_{\text{col}}} = 0.3$, $r_{\min_{\text{layers}}} = 0.7$, $r_{\max_{\text{layers}}} = 0.999$. Это даёт колонке 0, слою 0 горизонт памяти $\tau \approx 1$ шаг (быстрая реакция), а колонке $C-1$, слою $L-1$ — $\tau \approx 1000$ шагов (медленная интеграция). Никаких дополнительных параметров не требуется.

Уровень сообщений Хопфилда.. Межколоночный уровень сообщений использует современные сети Хопфилда с обучаемой температурой β на голову:

$$M_t = \text{softmax}(\beta \cdot Q_t^\top K_t) V_t$$

β инициализируется как $\log\left(\frac{1}{\sqrt{d_k}}\right)$ и обучается отдельно для каждой головы внимания.

Память записи с гейтом удивления (GridRNN-ЕМА)

Для частично наблюдаемого ОС агент должен записывать избирательно. Мы дополняем колонки Grid RNN матрицей быстрых весов с дельта-правилом (Ва и др., 2016) и **гейтом удивления**.

ЕМА-сигнал удивления.. Для каждой колонки c на слое l пусть y_t — постсмешанное состояние. Вычисляем ошибку предсказания:

$$k_t = \frac{W_k y_t}{\|W_k y_t\|_2}$$

$$\text{err}_t = v_t - W_t^\top k_t$$

ЕМА квадрата ошибки отслеживает **удивление**:

$$m_t = \beta_{\text{ema}} m_{t-1} + (1 - \beta_{\text{ema}}) \text{mean}(\|\text{err}_t\|^2)$$

$$\alpha_t = \frac{m_t}{\text{sg}(\max_B(m_t)) + \varepsilon}$$

где $\text{sg}(\cdot)$ обозначает остановку градиента. $\alpha_t \in [0, 1]$: элемент батча с наибольшим недавним удивлением всегда получает $\alpha_t \approx 1$ (активная запись), тогда как элементы со знакомыми входами стремятся к нулю.

Обновление по дельта-правилу..

$$W_{t+1} = (1 - \lambda_t) \delta^l W_t + \alpha_t \frac{k_t \otimes \text{err}_t}{C}$$

где $\delta^l \in [0.95, 0.99]$ — поперслоное затухание, λ_t — адаптивное забывание пропорциональное заполненности матрицы.

Модель	Параметры	Асс	Асс++
GRU (однопоточный)	2.1M	0.50	0.15
GridRNN (ячейки GRU)	2.1M	0.960	0.917
GridRNN-LRU	2.1M	0.849	0.694
HopfieldGridLRU	2.1M	0.967	0.932
GridHarmonic	2.1M	0.849	0.696

Таблица 2. Результаты SDQ-Hard. Асс — общая точность по токенам; Асс++ — точность только на трудных токенах запроса. GridRNN и HopfieldGridLRU: среднее за последние 10M шагов (стабильно). GridHarmonic: лучший чекпоинт на шаге 95M (пик перед началом осцилляций).

Эксперименты

Мы оцениваем три варианта Grid RNN по сравнению с базовыми линиями GRU на трёх бенчмарках. Все модели обучаются на одном GPU (RTX 3050). Бюджеты параметров согласуются в пределах бенчмарков на уровне $\approx 2.1M$.

Store-Distract-Query (SDQ)

Задача.. SDQ (Morad и др., 2023) представляет последовательности токенов из словаря из 65 символов (50 событий хранения, 10 отвлекающих, 5 типов запросов). Событие **хранения** предъявляет пару ключ-значение (k, v) , которую модель должна связать для последующего извлечения. Токены **запроса** предъявляют ранее сохранённый ключ и требуют вывода правильного значения. Вариант **Hard** дополнительно учитывает общее количество сохранённых и запрошенных пар одновременно.

Результаты.. Таблица Таблица 2 показывает метрики точности после $\sim 1B$ шагов обучения. Асс++ — точность **только** на трудных токенах запроса.

GridRNN (колонки GRU) значительно превосходит однопоточный GRU: Асс++ улучшается с ≈ 0.15 до 0.917 (+77 п.п. абсолютных). Это демонстрирует, что **структура колонок сама по себе** — без специализированных механизмов памяти — обеспечивает решающее преимущество. Сетка создаёт структурную дифференциацию, которой однопоточный GRU не может достичь вне зависимости от ширины или глубины.

HopfieldGridLRU (0.967 Асс, 0.932 Асс++) дополнительно улучшает вариант с колонками GRU, объединяя стабильность спектральной памяти (спектральная сетка LRU) с острым поиском Хопфилда для маршрутизации межколоночной информации.

GridHarmonic (0.849 Асс++) в текущем виде уступает HopfieldGridLRU. Две известные проблемы: (1) нормы представлений колонок неограниченно растут в верхних слоях после 100M шагов; (2) метрика Асс/store

Модель	RepeatFirst	HigherLower
GRU (базовая линия)	ожидается	ожидается
GridRNN (GRU)	ожидается	ожидается
GridRNN-EMA	0.95 [†]	0.41 [†]

Таблица 3. Результаты POPGym (суммарный возврат эпизода, шкала $\{-1, 1\}$). [†]Результаты на 74M/200M шагов (37% обучения). Полные результаты за 200M шагов и сравнение с GRU ожидаются.

возвращает NaN из-за ошибки в маскировке, что делает качество операции хранения непроверяемым.

Бенчмарк MIKASA/POPGym

Задача.. POPGym (Morad и др., 2023) предоставляет набор частично наблюдаемых сред, тестирующих четыре типа памяти. Мы оцениваем на двух задачах MIKASA (Cherepanov и др., 2025):

- **RepeatFirst** (объектная память): агент наблюдает N символов и должен воспроизвести первый в конце эпизода (51 шаг).
- **HigherLower** (последовательная память): предсказывать, будет ли следующая карта выше или ниже текущей.

Все модели обучаются с PPO (Schulman и др., 2017), $\gamma = 0.99$, $\lambda_{GAE} = 0.95$, 64 параллельных среды, длина роллаута 32, до 200M шагов среды.

Результаты.. GridRNN-EMA достигает суммарного возврата ≈ 0.95 на RepeatFirst, приближаясь к теоретическому максимуму 1.0 уже при 37% обучения. Это указывает на то, что ЕМА-гейт удивления успешно определяет событие первого наблюдения (единственное, которое стоит записать) и надёжно хранит его против 50 последующих отвлекающих факторов.

Кривая обучения GridRNN-EMA.. На 37% обучения возврат RepeatFirst (≈ 0.95) уже близок к оптимуму. Сходимость на этой среде типично наступает между 30–50M шагами; оставшиеся 126M шагов ожидаемо скорее консолидируют, чем существенно улучшают результат.

Нестабильность LRU под PPO.. Две дополнительные модели были остановлены досрочно на 15M шагах с отрицательным суммарным возвратом. В GridRNN-LRU энтропия политики H обрушилась с 1.1 до 0.19 бит в первые 10M шагов — необратимая детерминизация. В HopfieldGridLRU H оставалась выше (1.18 бит), но обучение осциллировало. Оба поведения указывают на структурную несовместимость линейной рекуррентности LRU со стандартным коэффициентом энтропии PPO (0.01).

Языковое моделирование

Постановка.. Оцениваем на двух посимвольных наборах данных: **text8** (100M символов, алфавит 27) и **Shakespeare** (1.1M символов, алфавит 65). Модели обу-

Модель	Параметры	Text8 BPC	Шекспир BPC
Transformer-XL (Dai и др., 2019)	277M	1.06	—
SHA-RNN (Merity, 2019)	53M	1.07	—
AWD-LSTM (Merity и др., 2018)	24M	1.19	—
GRU (наш)	2.1M	2.09	1.95
GridRNN (GRU)	2.1M	—	1.72
HopfieldGridRNN	2.1M	—	1.68
GridHarmonic	2.1M	1.68	—

Таблица 4. Посимвольное языковое моделирование (бит на символ, BPC; меньше — лучше). Опубликованные базовые линии используют модели в 12–130 × больше. Все варианты Grid RNN используют ≈2.1M параметров.

чаются с BPTT, 128 параллельными потоками, с расписанием сброса по убывающей вероятности. Все модели Grid RNN используют ≈ 2.1M параметров.

GridHarmonic достигает 1.68 BPC на text8 с 2.1M параметрами против AWD-LSTM при 1.19 BPC с 24M (в 12× больше). Цель — не закрыть абсолютный разрыв с SOTA, а продемонстрировать устойчивое улучшение в пределах фиксированного бюджета параметров. GridHarmonic улучшает однопоточный GRU на 0.41 BPC (19.6% относительных).

Динамика обучения.. BPC GridHarmonic монотонно убывает с 2.83 на 5M шагах до 1.68 на 270M — без дивергенции и осцилляций, в отличие от SDQ. Контекст T расширяется с 100 до 239 токенов по мере улучшения BPC (расписание учебной программы). Остаточная скорость улучшения на 250–270M составляет 0.010 BPC/50M шагов, что предполагает дальнейшие улучшения при продолжении обучения.

Диагностика памяти.. GridHarmonic показывает интерпретируемые внутренние метрики модуля SurpriseDelta. На 270M шагах (text8): нормы матрицы весов монотонно растут по слоям ($\|W\|_F$: L0=1.07, L1=1.53, L2=1.90), подтверждая, что более глубокие слои — с более медленными спектральными радиусами ($r_{\max}$ до 0.999) — накапливают более устойчивые ассоциации. Гейт записи α_t остаётся в (0.85, 0.89) по всем слоям без насыщения. Заполненность матрицы не превышает 3% на всех слоях.

Колонки	Слой	N	Асс	Асс++
2	1	115	0.734	0.494
4	3	128	0.960	0.917

Таблица 5. Абляция: число колонок и глубина при фиксированных ≈2.1M параметрах (SDQ-Hard). Увеличение с 2 до 4 колонок (+2 слоя) даёт +42 п.п. Асс++.

Анализ специализации колонок

Центральное утверждение статьи — колонки спонтанно специализируются. Мы описываем план количественного анализа, реализованный в инструменте визуализации.

Attention heatmap по фазам задачи (SDQ).. Для модели Grid RNN логируем матрицы внимания $A \in \mathbb{R}^{C \times C}$ (источник × получатель) отдельно в фазах хранения, отвлечения и запроса. Ожидаемый паттерн: в фазе хранения поток внимания направлен от колонки 0 (вход) к остальным; в фазе запроса — обратно. Аналогичный анализ для базовой линии GRU: скрытое состояние разбивается на C равных сегментов, для каждого вычисляется $MI(h_{\text{seg}}, \text{phase})$.

РСА скрытых состояний.. После обучения прогоняем тестовые эпизоды и собираем $\{h_t^{L,c}\}$ для каждой колонки c и токена t . PCA до 2 мерений: раскраска по фазе задачи (хранение / отвлечение / запрос). Ожидаем: состояния разных колонок кластеризуются по фазам, подтверждая ролевую специализацию.

Активационные нормы на MIKASA (RepeatFirst).. Для каждой колонки отслеживаем $|h^{L,c}|_2$ по шагам эпизода. В задаче RepeatFirst (запомнить первый объект) ожидаем: одна колонка поднимает норму в момент $t = 0$ и удерживает её до конца, остальные колонки остаются низкими.

Взаимная информация колонки и фазы.. $MI(h^{L,c}, \text{phase})$ для каждой колонки c : аппроксимируем дискретизацией нормы $|h^{L,c}|_2$. Для Grid RNN ожидаем неравномерное распределение MI по колонкам (разные колонки специализированы на разных фазах). Для GRU-сегментов ожидаем близкое к равномерному. Все описанные метрики логируются в AIM как изображения через каждые N млн. шагов (параметр `vis_interval` в конфигурации).

Абляция: число колонок

Чтобы изолировать влияние ширины сетки, сравниваем две конфигурации GridRNN с одинаковым бюджетом параметров (≈ 2.1M) на SDQ-Hard:

Переход от 2 колонок / 1 слоя к 4 колонкам / 3 слоям при сохранении числа параметров даёт +22.6 п.п. Асс (0.734 → 0.960) и +42.3 п.п. Асс++ (0.494 → 0.917). Это подтверждает, что число колонок — основной драйвер производительности, а не размер скрытого состояния.

Обсуждение

Почему структура колонок помогает? Сетка создаёт структурную асимметрию: колонка 0 всегда получает данные; другие колонки должны извлекать информацию через уровень сообщений. Эта асимметрия принуждает колонки к дифференцированным ролям. Обучаемые идентификаторы колонок позволяют вниманию выбирать исходные колонки по идентичности, обеспечивая возникновение согласованных политик маршрутизации.

Когда Grid RNN терпит неудачу? HopfieldGridLRU и GridRNN-LRU оба терпят неудачу в раннем ОС из-за коллапса энтропии под PPO. Более чистая абляция уровня сообщений Хопфилда потребует варианта с колонками GRU с сообщениями Хопфилда против стандартного MHA; сейчас GridRNN-LRU vs. HopfieldGridLRU смешивает тип ячейки (GRU vs. LRU) с типом сообщений.

Ограничения. (1) Результаты GridHarmonic на SDQ представлены на пике (95М шагов), а не при сходимости — модель осциллирует впоследствии из-за неограниченного роста нормы колонок. (2) Сравнение Hopfield/LRU в Таблице 1 смешано. (3) Базовая линия GRU на MIKASA ещё не получена — критический пробел перед публикацией этого раздела.

Полный перебор $C \in \{2, 3, 4, 5\}$ колонок при фиксированных параметрах необходим для чёткого количественного определения эффекта числа колонок независимо от глубины; текущая абляция охватывает только две точки.

Заключение

Мы представили модульный фреймворк Grid RNN — двумерную рекуррентную структуру $L \times C$, в которой колонки обмениваются информацией через контентно-ориентированное межколоночное внимание с обучаемыми идентификаторами. В отличие от предшественников (MD-RNN, Grid LSTM), использующих фиксированные позиционные связи, и от подходов с внешней памятью (NTM, DNC, fast weights), расширяющих однопоточный каркас, фреймворк реализует специализацию **через пространственное разделение рекуррентной динамики**. Ключевое свойство — модульность: ячейку колонки и уровень межколоночных сообщений можно заменять независимо, адаптируя модель к разным задачам без перестройки архитектуры. Три конфигурации на трёх бенчмарках неизменно и существенно превосходят однопоточные GRU-базовые линии: HopfieldGridLRU достигает 96.7% на SDQ-Hard; GridRNN-EMA достигает суммарного возврата ≈ 0.95 на POPGym RepeatFirst при 37% обучения; GridHarmonic достигает 1.68 BPC на text8 с 2.1M параметрами.

Будущая работа завершит оценку на MIKASA (включая HigherLower, MultiarmedBandit и более сложные уровни сложности), обеспечит полное сравнение с опубликованными базовыми линиями POPGym, исследует регуляризацию энтропии для улучшения сходимости ячеек LRU под PPO, проведёт систематическую абляцию числа колонок, а также дополнит эксперименты визуализацией специализации колонок (анализ внимания по фазам задачи и PCA скрытых состояний).

Список литературы

- Ba, J., Hinton, G. E., Mnih, V., Leibo, J. Z., & Ionescu, C. (2016). Using Fast Weights to Attend to the Recent Past. *Advances in Neural Information Processing Systems (NeurIPS)*, 29.
- Cherepanov, E., Kuznetsov, S., & Makarov, I. (2025). MIKASA: Memory-Intensive Knowledge Acquisition and Structured Assessment for Reinforcement Learning Agents. *arXiv preprint arXiv:2501.14346*.
- Cho, K., Merriënboer, B. van, Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.
- Chung, J., Ahn, S., & Bengio, Y. (2017,). Hierarchical Multiscale Recurrent Neural Networks. *5th International Conference on Learning Representations (ICLR)*.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2978–2988.
- Danihelka, I., Wayne, G., Uria, B., Kalchbrenner, N., & Graves, A. (2016). Associative Long Short-Term Memory. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1986–1994.
- Graves, A., Fernandez, S., & Schmidhuber, J. (2007). Multi-Dimensional Recurrent Neural Networks. *Proceedings of the 17th International Conference on Artificial Neural Networks (ICANN)*, 549–558.
- Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. *arXiv preprint arXiv:1410.5401*.
- Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., Colmenarejo, S. G., Grefenstette, E., Ramalho, T., Agapiou, J., Badia, A. P., Hermann, K. M., Zwols, Y., Ostrovski, G., Cain, A., King, H., Summerfield, C., Blunsom, P., Kavukcuoglu, K., & Hassabis, D. (2016). Hybrid computing using a neural network with dynamic external memory. *Nature*, 538, 471–476.

- Hausknecht, M., & Stone, P. (2015,). Deep Recurrent Q-Network. *AAAI Workshop on Deep Learning and Artificial Intelligence (AAAI-DL)*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Kalchbrenner, N., Danihelka, I., & Graves, A. (2016,). Grid Long Short-Term Memory. *3rd International Conference on Learning Representations (ICLR)*.
- Li, S., Li, W., Cook, C., Zhu, C., & Gao, Y. (2018). Independently Recurrent Neural Network (IndRNN): Building A Longer and Deeper RNN. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 5457–5466.
- Merity, S. (2019). Single Headed Attention RNN: Stop Thinking With Your Head. *arXiv preprint arXiv:1911.11423*.
- Merity, S., Keskar, N. S., & Socher, R. (2018,). Regularizing and Optimizing LSTM Language Models. *6th International Conference on Learning Representations (ICLR)*.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous Methods for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1928–1937.
- Morad, S. D., Kortvelesy, R., Bettini, M., Liwicki, S., & Prorok, A. (2023,). POPGym: Benchmarking Partially Observable Reinforcement Learning. *11th International Conference on Learning Representations (ICLR)*.
- Mujika, A., Meier, F., & Steger, A. (2017). Fast-Slow Recurrent Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
- Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., & De, S. (2023). Resurrecting Recurrent Neural Networks for Long Sequences. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 26670–26698.
- Qin, Z., Yang, S., Sun, W., Shen, X., Li, D., Li, W., & Zheng, Y. (2024,). HGRN2: Gated Linear RNNs with State Expansion. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Qin, Z., Yang, S., Zhong, W., He, X., Wu, F., Wang, Y., Sun, W., Yin, S., Zhang, Z., Zheng, Y., Zhao, X., & Liu, Y. (2023). HGRN: Hierarchically Gated Recurrent Neural Network for Sequence Modeling. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.
- Ramsauer, H., Schäfl, B., Lehner, J., Seidl, P., Widrich, M., Gruber, L., Holzleitner, M., Pavlovic, M., Sandve, G. K., Greiff, V., Kreil, D., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2021,). Hopfield Networks Is All You Need. *9th International Conference on Learning Representations (ICLR)*.
- Schlag, I., Irie, K., & Schmidhuber, J. (2021). Linear Transformers Are Secretly Fast Weight Programmers. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 9355–9366.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017,). *Proximal Policy Optimization Algorithms*.
- Szelogowski, M. (2025). Hebbian Memory-Augmented Recurrent Networks: Engram Neurons in Deep Learning. *arXiv preprint arXiv:2501.07108*.
- Zhang, Y., Yang, S., Zhu, R., Qin, Y., Ma, Y., Zhang, J., Li, D., & Qin, Z. (2024). Gated Slot Attention for Efficient Linear-Time Sequence Modeling. *Advances in Neural Information Processing Systems (NeurIPS)*, 37.